14. Debugging and Exception Handling

Exam Objectives

1. Identify syntax and logic errors
2. Use exception handling
3. Handle common exceptions thrown
4. Use try and catch blocks

14.1 Create try-catch blocks and determine how exceptions alter program flow

14.1.1 Java exception handling

Exceptions are for managing exceptional situations. For a file copy program, the normal course of action could be - open file A, read the contents of file A, create file B, and write the contents to file B. But what if the program is not able to open file A? What if the program is not able to create or write to file B? There could be many reasons for such failure such as a typo in the file name, lack of permission, no space on disk, or even disk failure. Now, you don't expect these problems to occur all the time but it is reasonable to expect them some times. Since we don't expect these problems to occur regularly in normal course of operation, we call them "exceptional".

In exceptional situations, you may want to give the user a feedback about the error or you may even want to take an alternative course of action. For example, you may want to let the user to input another source file name if the input file is not found or another target location if the given location is out of space. Even if you don't want to take any special action upon such situations, you should at least want your program to end gracefully instead of crashing unexpectedly at run time. This means, you should provide a path for the program to take in such situations. One way would be to check for each situation before proceeding to copy. Something like this:

```
if(checkFileAccess(file1)){
    if(checkWritePermission(targetDirectory){
        //code for normal course of action
     }else{
        System.out.println("Unable to create file2");
    }
}else{
    System.out.println("Unable to read file1");
}
```

You can see where this is going. You will end up having a lot of if-else statements. Not only is it cumbersome to code, it will be a nightmare to read and maintain later on.

There is another serious problem with the above approach. It doesn't provide way to write code for situations that you don't even know about at the time of writing the code. For example, what if the user runs this code in an environment that requires file names to follow a particular format? So, now, you have two kinds of "exceptional situations". One that you know about at the time of writing the code, and one that you don't know anything about.

Java exception mechanism is designed to help you write code that covers all possible execution paths that a program may take - 1. path for normal operation 2. paths for known exceptional situations, and 3. path for unknown exceptional situations. Here is how the above mentioned program can be written:

```
try{
```

```
   //code for normal course of action
}catch(SecurityException se){
   //code for known exceptional situation
   System.out.println("No permission!");
}
catch(Throwable t){
   //code for unknown exceptional situations
   System.out.println("Some problem in copying: "+t.getMessage());
   t.printStackTrace();
}
```

Observe that the perspective is reversed here. In the if/else approach, you check for each exceptional situation and proceed to copy if everything is good, while in the try/catch approach, you assume everything is good and only if you encounter a problem, you decide what to do depending on the problem. The benefits of the try/catch approach are obvious. It provides a clean separation between code for normal execution and code for exceptional situations, which makes the code easier to read and maintain. It allows an alternative path to the program to proceed even in cases where the programmer has not anticipated the problem.

Exception handling in Java is not without its criticism. There has been a good amount of debate on what constitutes "exceptional situations" and what should be the right approach to handle such situations. My objective in this chapter is to teach you Java's approach to exception handling and so, I will not go into an academic discussion on whether it is good or bad as compared to other languages. However, it is a very important topic of discussion in technical interviews. I suggest you google criticism of Java's exception handling and compare it with C#'s.

14.1.2 Fundamentals of the try/catch approach

When you think of developing code as developing a component, you will realize that there are always two stakeholders involved - the provider/developer of the component and the client or the user of the component. For the client to be able to use the component, it is imperative for the provider to tell the client about "how" his component works. The how not only includes the input/output details of the component but also the information about exceptional situations, i.e., about situations the component knows may occur but does not deal with.

For example, let's say you are developing a method that copies the contents of one file to another and that this method is to be used by a developer in another team. You must convey to the other developer that you are aware of the Input/Output issues associated with reading and writing a file but you won't do anything about them. In other words, you must convey that your method will try its best to copy the file but if there is an I/O issue, it will abandon the attempt and let her know about the failure so that she can deal with it however she wants to. This is done through the use of a **"throws"** clause in method declaration:

```
public void copyFile(String inputPath, String outputPath) throws IOException {
   //code to copy file
```

```
}
```

In the above code, if the `copyFile` method encounters any I/O issue while copying a file, it will simply abort the copying and throw an `IOException` to the caller. If it does not encounter any I/O issue, the method will end successfully.

On the other side of the component is the user of that component, who uses the information provided by the component provider to develop her code. The user has to decide how she wants to handle the exceptional situation. If she believes that she has the ability to "resolve" the situation, she will put the usage of that component in a **"try"** block with an associated **"catch"** block that contains code for "resolution" of the problem. When that exceptional situation actually occurs, the control goes to the catch block instead of proceeding to the next statement after the method call that threw the exception.

If the user decides that she cannot handle the exceptional situation either, she propagates the exception to the caller of her component.

For example, a program that creates backup of a file may use the file copy program internally to copy a file. If the file copy program throws an `IOException`, the backup creator program may catch that exception and show a message to the user.

```
public void createBackup(String input) {
   String output  = input+".backup";
   try{
      copyFile(input, output);
      System.out.println("backup successful");
   }catch(IOException ioe){
      System.out.println("backup failure");
   }
}
```

If the `copyFile` method completes without any issue, the control will go to the `println` statement. If the `copyFile` method throws an `IOException`, the control will go to the catch block, thus, providing an opportunity to take a different course of action. Here, the code may show a failure message to the user. From the perspective of the developer of the backup program, showing the error message to the user is the resolution of the I/O problem. One can also write code to have the user specify another file or directory to create a backup.

An exception is considered "handled" when it is caught in a catch block. If the backup creator method doesn't know what to do in case the `copyFile` is not successful, then it should let the exception propagate to its caller by using a throws clause of its own:

```
public void createBackup(String input) throws IOException{
   String output  = input+".backup";
   copyFile(input, output);
   System.out.println("backup successful");
}
```

In the above code, if the `copyFile` method completes without any issue, the control will go to the `println` statement. But if the `copyFile` method throws an `IOException`, the `createBackup` method will end immediately and the caller will receive an `IOException`. Note that this will be

the same `IOException` that it receives from the `copyFile` method. This is how an exception propagates from one method to the other. There could be a long chain of method calls through which an exception bubbles up before it is handled. If an exception is not caught anywhere while it is bubbling up the call chain, it ultimately reaches the JVM code. Since the JVM has no idea about the business logic of the program that it is executing, it "handles" the exception by killing(aka terminating) the program (actually, the JVM kills only the thread that was used to invoke the chain of method calls but threading is not on the JFCJA exam, so you can assume for now that a program is composed of only one thread and killing of that thread is the same as killing the program).

14.1.3 Pieces of the exception handling puzzle

Java exception handling mechanism comprises several moving parts. What makes this mechanism a bit complicated is that you need to put each part in its right place to make the whole thing work. So, let me introduce these parts first briefly.

The `java.lang.Throwable` object -

`Throwable` is the root class of all exceptions. It captures the details of the program and its surroundings at the time it is created. Among other things, a `Throwable` object includes the chain of the method calls that led to the exception (known as the "stack trace") and any informational message specified by the programmer while creating that exception. This information is helpful in determining the location and the cause of the exception while debugging a program. You may have seen crazy looking output on the console containing method names upon a program crash. This output is actually the stack trace contained in the `Throwable` object.

`Throwable` has two subclasses - `java.lang.Error` and `java.lang.Exception`, and a huge number of grand child classes. Each class is meant for a specific situation. For example, a `java.lang.NullPointerException` is thrown when the code tries to access a null reference or an `java.lang.ArrayIndexOutOfBoundsException` is thrown when you try to access an array beyond its size. You can also create your own subclasses by extending any of these classes if the existing classes don't represent the exceptional situation appropriately. For example, an accounting application could define a `LowBalanceException` for a situation where more money is being withdrawn from an account than what the account has.

What is called **"exception"** (i.e. exception with a lower case 'e') in common parlance, is in reality is an object of class `java.lang.Throwable` or one of its subclasses.

One subclass of `java.lang.Exception` that is particularly important is `java.lang.RuntimeException`. `RuntimeException` and its subclasses belong to a category of exception classes called **"unchecked exceptions"**. You will see their significance soon.

The `throw` statement

The `throw` statement is used by a programmer to "throw an exception" or "raise an exception" explicitly. A programmer may decide to throw an exception upon encountering a condition that makes continuing the execution of the code futile. For example, if, while executing a method, you

find that the value of a required parameter is invalid, you may throw an `IllegalArgumentException` using a `throw` statement like this:

```
public double computeSimpleInterest(double p, double r, double t){
   if( t<0) {
      IllegalArgumentException iae = new IllegalArgumentException("time is less than
   0");
      throw iae;
   }
   //other code
}
```

Usually, an exception object is created only to be "thrown" and so, there is no need to store its reference in a variable. That is why it is often created and thrown in the same statement as shown below:

```
public double computeSimpleInterest(double p, double r, double t){
  if( t<0) throw new IllegalArgumentException("time is less than 0");
  //other code
}
```

Throwing an exception implies that the code has encountered an unexpected situation with which it does not want to deal. The code shown above, for example, expects the time argument to be greater than zero. For this code, time being less than zero is an unexpected situation. It does not want to deal with this situation (probably because the programmer is not sure what to do in this case) and so it throws an exception in such a situation. This also means that this method passes on the responsibility of determining what should be done in case time is less than zero to the user of this method.

Note that only an instance of Throwable (or its subclasses) can be thrown using the `throw` statement. You cannot do something like `throw new Object();` or `throw "bad situation";`

Explicitly throwing an exception using the throw statement is not the only way an exception can be thrown. JVM may also decide to throw an exception if the code tries to do some bad thing like calling a method on a null reference. For example:

```
public void printLength(String str){
    System.out.println(str.length());
}
```

If you pass `null` to the above method, the JVM will create a new instance of `NullPointerException` and throw that instance when it tries to execute `str.length()`.

The `throws` clause

Java requires that you list the exceptions that a method might throw in the `throws` clause of that method. This ties back to Java's design goal of letting the user know of the complete behavior of a method. It wants to make sure that if the method encounters an exceptional situation, then the method either deals with that situation itself or lets the caller know about that situation. The

`throws` clause is used for this purpose. It conveys to the user of a method that this method may throw the exception mentioned in the `throws` clause. For example:

```
public double computeSimpleInterest(double p, double r, double t) throws Exception{
  if( t<0) throw new Exception("time is less than 0");
  //other code
}
```

Now, anyone who uses the above method knows that this method may throw an exception instead of returning the interest. This helps the user write appropriate code to deal with the exceptional situation if it arises.

The `try` statement

A **try statement** gives the programmer an opportunity to recover from and/or salvage an exceptional situation that may arise while executing a block of code. A try statement consists of a **try block**, zero or more **catch blocks**, and an optional **finally block**. Its syntax is as follows:

```
try {
  //code that might throw exceptions
}catch(<ExceptionClass1> e1){
  //code to execute if code in try throws exception 1
}
catch(<ExceptionClass2> e2){
  //code to execute if code in try throws exception 2
}
catch(<ExceptionClassN> en){
  //code to execute if code in try throws exception N
}
finally{
  //code to execute after the try block and catch block finish execution
}
```

Note that curly braces for the `try`, `catch`, and `finally` blocks are required even if there is a single statement in these blocks (compare that to `if`, `while`, `do/while` and `for` blocks where curly braces are not required if there is only one statement in the block). Furthermore, a **try block** must follow with at least one **catch block** or the **finally block**. A try block that follows with neither a catch block nor a finally block will not compile.

The `try` statement is the counterpart of the throw statement because putting a piece of code within a try block signifies that the programmer wants do something in case that piece of code throws an exception. In other words, a try block lets the programmer deal with an exceptional situation as opposed to the throw statement, which lets the programmer avoid dealing with it.

While a try block contains code for normal operation of the program, a **catch block** is the location where the programmer tries to recover from an exceptional situation that arises in the try block. If the code in the try block throws an exception, the normal flow of execution in that try block is aborted (i.e. no further code in the try block is executed) and the

control goes to the catch block. Here, the programmer can take alternate approach to finish the processing of the method. For example, the programmer may decide to just show a popup message to the user about the exception and move on to the next statement after the try statement.

A catch block is associated with a **catch clause**, which specifies the exception that the catch block is meant to handle. For example, a catch block with the catch clause as `catch(IllegalArgumentException e)` is meant to handle `IllegalArgumentException` and its subclasses. Thus, this catch block will be executed only if the code in the try block throws an `IllegalArgumentException` or its subclass exception. You can specify any valid exception class (including `java.lang.Throwable`) in the catch clause.

A **finally block** is the location where the programmer tries to salvage the situation or control the damage so to say, without attempting to recover from an exception. For example, a program that tries to copy a file may want to close any open files irrespective of whether the copy operation is successful or not. The programmer can do this in the finally block. You may think of a finally block as the step where a car mechanic reassembles the parts back irrespective of whether he was able to fix the car or not!

Here is a method that calls the `computeSimpleInterest` method shown above within a try statement:

```
public void doInterest(){

   try{
      double interest = computeSimpleInterest(1000.0, 10.0, -1);
      System.out.println("Computed interest "+interest);
   }catch(Exception e){
      System.out.println("Problem in computing interest:"+e.getMessage());
      e.printStackTrace();
   }finally{
      System.out.println("all done");
   }

}
```

In the above code, the call to `computeSimpleInterest` throws an `IllegalArgumentException` because `t` is negative. Thus, the `println` statement after the method call is not executed. The exception is caught in the catch block because its catch clause, i.e., `catch(IllegalArgumentException )` matches with the exception that is thrown by the try block and the control goes to the first statement in this catch block. It prints the exception's message and the stack trace on the console. Finally, the control goes to the code in the finally block, where it prints `"all done"`. If you omit the catch block, the control will go directly to the finally block after the invocation of `computeSimpleInterest` method. After the execution of the code in the finally block, the caller of `doInterest` method will receive the same `IllegalArgumentException`.

Note that a finally block, if present, always executes irrespective of what happens in the

try block or the catch block. Even if the try block throws an exception and there is no catch block to catch that exception, the JVM will execute the finally block. It will throw the exception to the caller only after the finally block finishes. The only case where the finally block is not executed is if the code in the try or the catch block forces the JVM to shut down using a call to `System.exit()` method.

The following is a complete program that illustrates how an exception alters the normal program flow:

```
public class TestClass{

  public static void main(String[] args){
    TestClass tc = new TestClass();
    tc.doInterest();
  }

  public double computeSimpleInterest(double p, double r, double t){
     if( t<0) throw new IllegalArgumentException("time is less than 0");
     return p*r*t/100;
  }

  public void doInterest(){
     try{
        double interest = computeSimpleInterest(1000.0, 10.0, -1);
        System.out.println("Computed interest "+interest);
     }catch(IllegalArgumentException iae){
        System.out.println("Problem in computing interest:"+iae.getMessage());
        iae.printStackTrace();
     }finally{
        System.out.println("all done");
     }
  }

}
```

It generates the following output on the console :

```
Problem in computing interest:time is less than 0
java.lang.IllegalArgumentException: time is less than 0
        at TestClass.computeSimpleInterest(TestClass.java:8)
        at TestClass.doInterest(TestClass.java:14)
        at TestClass.main(TestClass.java:4)
all done
```

You should observe the following points in the above code:

1. The `return` statement in `computeSimpleInterest` is not executed because the previous statement throws an exception.

2. The `println` statement in the try block of `doInterest` is not executed because the call to `computeSimpleInterest` ends with an exception instead of a return value. Control goes to the `catch` block directly after the method call.

3. The `catch` block prints the details captured in the exception object. It shows the sequence of the method invocations in reverse order from the point where the `IllegalArgumentException` object was created. You should try removing the catch block and see the output.

4. Once the `catch` block is finished, the control goes to the `finally` block.

5. The `doInterest` method returns after the execution of the `finally` block.

6. There is no throws clause in `computeSimpleInterest` method even though it throws an exception. The reason will be clear in the next section where I talk about checked and unchecked exceptions.

14.2 Differentiate among checked, unchecked exceptions, and Errors

14.2.1 Checked and Unchecked exceptions

As discussed earlier, exceptions thrown by a method are part of the contract between the method and the user of that method. If the user is not aware of the exceptions that a method might throw, she will be blindsided during run time because her code would not be prepared to handle that exception. The `throws` clause of a method is meant to list all exceptions that the method might throw. If an exception is listed in the throws clause, the user of the method will know that she needs to somehow handle that situation.

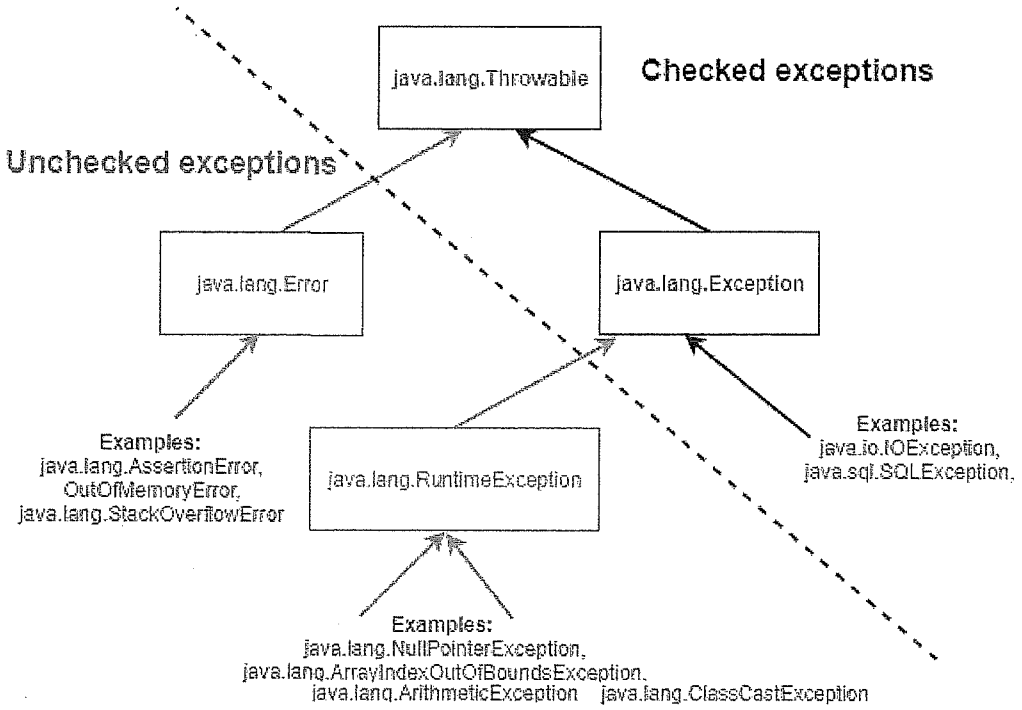
But what if a developer forgets to list an exception in the throws clause of a method? In that case, we are back to the same problem of blindsiding the user of that method at run time. This is where the compiler plays an important role. While compiling a method, the compiler checks for the possibility of any exception that might get thrown by the method to the caller. If that exception is not listed in the throws clause of that method, it refuses to compile the method.

This sounds like a good solution, but the problem here is that the compiler does not check for all kinds of exceptions thrown by a piece of code. It checks for only a certain kind of exceptions called **"checked exceptions"** and forces you to list the only those exceptions that belong to this category of exceptions in the throws clause.

Exceptions that do not belong to the category of checked exceptions are called **"unchecked exceptions"**. They are called unchecked because the compiler doesn't care whether a piece of code throws such an exception or not. Listing unchecked exceptions in the throws clause is optional.

Finding out whether an exception is a checked exception or not is easy. Java language designers have postulated that any exception that extends `java.lang.Throwable` but does not extend `java.lang.RuntimeException` or `java.lang.Error` is a checked exception. The rest (i.e. `java.lang.Error`, `java.lang.RuntimeException`, and their subclasses) are unchecked.

The following figure illustrates this grouping of exceptions.



Checked and unchecked exceptions

Note that Java has a deep rooted exception class hierarchy, which means there are several classes, subclasses, and subclasses of subclasses in the tree of exceptions. So just because an exception is a `RuntimeException`, does not mean that that exception directly extends `RuntimeException`. It could even be a grand child of `RuntimeException`. For example, `ArrayIndexOutOfBoundsException` actually extends `IndexOutOfBoundsException`, which in turn extends `RuntimeException`. Similarly, just because an exception is a checked exception does not mean that it directly extends Exception. It may extend any subclass of `Exception` (except `RuntimeException`, of course).

Rationale behind checked and unchecked exceptions

Recall our discussion on exceptional situations where I talked about two kinds of exceptional situations - ones that a developer knows about and ones that a developer doesn't expect to occur at all. While a developer may want to provide an alternate path of execution in case of a situation that is known to occur but there is no point in providing an alternate path of execution for a situation that is never expected to occur.

For example, if a piece of code tries to write to a file, the developer may want to take a different approach if he is not able to write to the file system. But if the data array that it is trying to write is null, there is nothing much he can do. Such unexpected situations occur mostly due to badly written code. In other words, if a code gets itself into an unexpected situation, it is most likely because of a programming error, i.e., a bug in the code. Such issues should be fixed during development itself. But if they do occur in production, they should rather be handled at a much higher level than at the component level.

Unchecked exceptions are for such **unexpected** situations. Java language designers believe that unchecked exceptions need not be declared in a method declaration because there is nothing to gain by forcing the caller to catch them. Only checked exceptions need to be declared because the caller of the method may have a plan to recover from them. There are two kinds of unchecked exceptions - exceptions that extend `java.lang.RuntimeException` (aka **runtime exceptions**) and exceptions that extend `java.lang.Error` (aka **errors**).

Runtime exceptions and errors

Characterizing a situation as expected or unexpected is a design decision that depends on the business purpose of the method. One method may expect to receive a null argument and may work well if it gets a null argument, while another may not expect a null argument and may end up throwing a `NullPointerException` when it tries to access that null. In the second case, passing a null to that method would be considered a bug in the code, which must be identified and fixed during testing. Such exceptions that signify the presence of a bug in the code are categorized as **runtime exceptions**.

Here, the word runtime in RuntimeException does not imply that only exceptions that extend RuntimeException can be thrown at run time. All exceptions are thrown only when the program is executed, i.e., at run time. The word **runtime** refers to the fact that the developer comes to know of the occurrence of the situation that results in a RuntimeException only when the program is run, i.e., during run time. Had the programmer anticipated the occurrence of that situation during compile time, he would have fixed the code, and in which case the exception would not have been thrown upon running the program. For the same reason, runtime exceptions are usually **not thrown explicitly** using the `throw` statement. Indeed, why would you write the code to throw an exception if you don't even expect that situation to occur? The JVM throws runtime exceptions on its own when it encounters an unexpected situation. For example, if you try to access a null reference, the JVM will throw a `NullPointerException`, or if you try to access an array beyond its size, the JVM will throw an `ArrayIndexOutOfBoundsException`. It is possible to recover from runtime exceptions but ideally, since they indicate bugs in the code, you should not attempt to catch them and recover from them. A well written program should not cause the JVM to throw runtime exceptions.

The case of **errors** is similar to **runtime exceptions**. The difference is that errors are reserved for situations where the operation of the JVM itself is in jeopardy. For example, a badly written code may consume so much memory that there is no free memory left. Once that happens, the JVM may end up throwing `OutOfMemoryError`. Similarly, a bad recursion may cause `StackOverflowError` from which no recovery is possible. Errors signify serious issues in the interaction between the code and the JVM and are thrown exclusively by the JVM. It is never a good idea to throw them explicitly or to try to recover from them because the code will likely not work as expected anyway once the JVM starts throwing Errors.

Although the exam does not focus on the reason for categorizing exception between checked and unchecked exceptions, it is actually a very important topic to understand for a professional developer. You should also compare Java's exception handling with C#'s.

Identifying exceptions as checked or unchecked

Java follows a convention in naming exception classes. This convention is sometimes helpful in determining the kind of exception you are dealing with. The name of any class that extends `Error` ends with **Error** and the name of any class than extends `Exception` ends with **Exception**. For example, `OutOfMemoryError` and `StackOverflowError` are Errors while `IOException`, `SecurityException`, `IndexOutOfBoundsException` are Exceptions.

However, there is no way to distinguish between unchecked exceptions that extend `RuntimeException` and checked exceptions just by looking at their names. It is therefore, important to memorize the names of a few important runtime exception classes, namely - `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `ClassCastException`, and `SecurityException`.

14.2.2 Commonly used exception classes

The Java standard library contains a huge number of exception classes but the exam expects you to know about only a few of them, which I will cover now. These exception classes are important because you will encounter them quite often while working with Java.

I have omitted the package name from the classes below for brevity but note that all of the following exception classes belong to the `java.lang` package.

Common Exceptions that are usually thrown by the JVM

1. `ArithmeticException` extends `RuntimeException`
   The JVM throws this exception when you try to divide a number by zero. Example :

```
public class X {
   static int k = 0;
   public static void main(String[] args){
     k = 10/0; //ArithmeticException
   }
}
```

2. `IndexOutOfBoundsException` extends `RuntimeException`
   This exception is a common superclass of exceptions that are thrown where an invalid index is used to access a value that supports indexed access.

   For example, the JVM throws its subclass `ArrayIndexOutOfBoundsException` when

you attempt to access an array with an invalid index value such as a negative value or a value that is greater than the length (minus one, of course) of the array. Methods of String class throw another of its subclass `StringIndexOutOfBoundsException` when you try to access a character at an invalid index.

Example :

```
int[] ia = new int[]{ 1, 2, 3}; // ia is of length 3
System.out.println(ia[-1]); //ArrayIndexOutOfBoundsException
System.out.println(ia[3]); //ArrayIndexOutOfBoundsException

System.out.println("0123".charAt(4)); //StringIndexOutOfBoundsException
```

3. `NullPointerException` extends `RuntimeException`
   The JVM throws this exception when you attempt to call a method or access a field using a reference variable that is pointing to null. Example :

```
String s = null;
System.out.println(s.length()); //NullPointerException because s is null
```

14.3 Use try and catch blocks

14.3.1 Creating a method that throws an exception

Now that you know about the throws clause and the types of exceptions, let us look at the rules for creating a method that throws an exception. Actually, there is just one rule - If there is a possibility of a **checked exception** getting thrown out of a method, then that exception or its superclass exception must be declared in the **throws clause** of the method. The following are examples of a few valid methods that illustrate this rule:

1. --

```
void process(int x) throws Exception{
    if(x == 2) throw new Exception(); //throws Exception only if x==2
    else return;
}
```

   It doesn't matter whether the code throws an exception every time it runs or only some times. If there is a path of execution in which an exception will be throw, the method must list that exception in the throws clause.

2. --

```
void process() { //no throws clause necessary
    if(someCondition) throw new RuntimeException();
    else throw new Error();
}
```

`RuntimeException` and `Error` are **unchecked exceptions** and are therefore, exempt from being declared in the throws clause. Declaring them in the throws clause is valid though.

3. --

```
void process() throws Exception{ //declaring Exception in the throws clause even
    though it is not thrown by the method body
   System.out.println("hello");
}
```

A method can declare any exception in its throws clause irrespective of whether that method actually throws that exception or not. It is sometimes useful to "future-proof" a method by declaring Exception in the throws clause if you believe that the method's implementation may change later. By declaring Exception in the throws clause, the users of the method will not have to change their code if the method actually starts throwing Exception or any of Exception subclasses later.

4. --

```
void process1() {
    try{
       if(someCondition) throw new Exception(); //will be caught by the catch
    block
       else return;
    }catch(Exception e){

    }
}
```

The requirement to list an exception in the throws clause is applicable only when an exception is thrown out of the method to the caller. If the code inside a method throws an exception but that exception is caught within the method itself, there is no need to declare it in the throws clause of the method. Remember that an exception can only be caught by a catch block and not by a finally block. Therefore, the throws clause is necessary in the following method:

```
void process2() throws Exception{
    try{
       if(someCondition) throw new Exception();
       else return;
    }finally {
       System.out.println("in finally"); //will be executed but the exception is
    not caught here
    }
}
```

Although not important for the exam, deciding which exception to throw and which to declare is an important matter.

Declaring exceptions

As shown in point number 8 above, declaring a broad exception class in the throws clause is an easy way to get rid of any compilation errors with the method if the method throws different kinds of exceptions from different parts of its code. However, this is considered a bad practice because this burdens the user of the method with dealing with a broad range of exceptions. On the other hand, listing specific exception classes individually restricts the future modifiability of a method because throwing new exceptions later will break other people's code. It is recommended to be balanced in your approach towards listing exceptions in the throws clause. You should try to be only as specific as is possible without compromising the modifiability of the method. For example, I/O related methods may encounter different kinds of issues while reading a file. It may not be possible to identify all such issues while writing the method. New issues may be discovered later and you may have to modify your method to accommodate those. Therefore, it is better to declare a common superclass `IOException` in the throws clause instead of individual subclasses such as `FileNotFoundException` or `EOFException`.

Throwing exceptions

You should always throw the most specific exception possible. For example, if you don't find the file while trying to open it, you should throw a `FileNotFoundException` instead of an `IOException` or `Exception`. By throwing the most specific exception, you give more information to the caller about the problem. This helps the caller in determining the most suitable resolution of the problem.

14.3.2 Invoking a method that throws an exception

The rules for invoking a method that throws an exception are similar to the ones for creating a method that throws an exception. As far as the compiler is concerned, there is no difference between using a `throw` statement to throw an exception and invoking a method that throws an exception to throw an exception. In both the cases, the compiler forces you to either declare the exception in your `throws` clause or handle the exception using a `try/catch` block. Of course, as discussed before, the compiler is only concerned with **checked exceptions**. The following example illustrates this point:

```
public class TestClass{
  public static void message() throws Exception{
     if(true) throw new Exception();
     else return;
  }
  public static void main(String[] args) throws Exception {
      message();
```

```
  }
}
```

In the above code, the method `message()` throws an `Exception` using the `throw` statement. Since `Exception` is a checked exception, the compiler forces it to be declared in the `throws` clause. On the other hand, instead of throwing an exception explicitly using the `throw` statement, `main` invokes `message`. But the compiler knows that invoking `message` may result in an `Exception` being thrown (because it is mentioned in the `throws` clause of `message`) and so the compiler forces `main` to declare `Exception` in its `throws` clause as well. The other option for `main` is, of course, to handle the exception itself:

```
public static void main(String[] args) { //no throws clause necessary
      try{
         message();
      }catch(Exception e){
         e.printStackTrace();
      }
  }
```

> If a method doesn't want to catch the exception then it must declare that exception (or its superclass) in its `throws` clause. This is no different from the rule that you have seen before while creating a method that throws an exception. For example, assuming that the method `message` throws `Exception`, here are valid throws clauses for a method that invokes `message` :

```
//declare the same exception class
  public static void callMessage() throws Exception {
         message();
  }

  //declare a super class of the exception class thrown by message
  public static void callMessage() throws Throwable {
         message();
  }
```

Of course, `callMessage` is not limited to throwing just the exceptions declared in the throws clause of `message`. It can add its own exceptions to the throws clause irrespective of whether the code inside the method throws them or not.

To catch or to throw

The decision to catch an exception or to let it propagate to the caller depends on whether you can resolve the problem that resulted in the exception being thrown or not. Consider the following code for a method that computes simple interest:

```
public static double computeInterest(double p, double r, int t) throws Exception{
   if(t<0) throw new Exception("t must be > 0");
   else return p*r*t;
}
```

and the following code that uses the above method:

```
public static void main(String[] args){
   double interest = 0.0;
   try{
      computeInterest(100, 0.1, -1);
  }catch(Exception e){
  }
  System.out.println(interest);
}
```

Upon execution, the main method prints interest as `0.0` even though the `computeInterest` method did not really compute interest at all. It threw an exception because t was less than 0. However, as a user of the program, you won't know that there was actually a failure during the computation of interest.

While the `computeInterest` method did its job of telling the `main` method of a problem in computation by throwing an exception, the `main` method sweeped this problem silently under the rug by using an empty `catch` block. This is called **"swallowing the exception"** and is a bad practice.

The purpose of a `catch` block is to resolve the problem and not to cover up the problem. By covering up the problem, the program keeps running but starts producing illogical results. Ideally, `main` should not have caught the exception but declared the exception in its `throws` clause because it is in no position to resolve the problem. It would have been appropriate for a program with GUI to catch the exception and ask the user to input valid arguments.

Sometimes, it becomes necessary to catch an exception even though no resolution is possible at that point. The right approach in such a case is to log the exception to the console so that the program can be easily debugged later by inspecting the logs.

```
public static void main(String[] args){
   double interest = 0.0;
   try{
      computeInterest(100, 0.1, -1);
  }catch(Exception e){
     e.printStackTrace();
     //or System.out.println(e);
  }
  System.out.println(interest);
}
```

14.4 Exercise

1. Create a method named `countVowels` that takes an array of characters as input and returns the number of vowels in the array. Furthermore, the method should throw a checked exception if the array contains the letter `'x'`.

2. Invoke the `countVowels` method from main in a loop and print its return value for each command line argument. Observe what happens in the following situations: there is no command line argument, there are multiple arguments, there are multiple arguments but the first argument contains an `'x'`. (Use String's `toCharArray` method to get an array of characters from the string.)

3. Ensure that your main method prints the number of vowels in other command line arguments even if one argument contains an `'x'`.

4. Pass null to the `countVowels` method and observe the output.

5. Modify `countVowels` method to throw an `IllegalArgumentExcception` if it is passed a `null`.

6. Modify `countVowels` method to return `-1`, if the input array is `null` and 0, if the input array length is less than `10`. Do not use an `if` statement.

7. Create a method `int sum(int[] values, int start, int end)` that throws an `IllegalArgumentException` when passed an array of length 0, a `NullPointerException` when passed a `null`, and `ArrayIndexOutOfBoundsException` when `start` and `end` do not fall within the range of the given array. It should return the sum of the values in the array from `start` to `end` but must throw `Exception` when sum is 0.

8. Invoke the sum method created above from main. Wrap the call in an appropriate try/catch statement. Print two different messages on console depending upon whether the call results in a checked exception or an unchecked exception. Propagate all exceptions up the call chain.

Index

Symbols

A

D

E

J

K

L

M

N

O

T

U

Made in the USA
Las Vegas, NV
27 January 2026